

Chapter 5. Machine Learning Model Deployment

So far, we've explored the individual stages of building and deploying a ML model: data preprocessing, feature engineering, model training, evaluation, and serving. In this chapter, we'll shift our focus to orchestration and look at how to connect these steps into an automated pipeline that runs reliably in production.

We'll walk through how to structure end-to-end ML workflows on Databricks, including platform components, workspace design, permission management, and deployment strategies. By the end of the chapter, you'll understand how to design your ML pipeline using best practices that are suited to the Databricks environment.

Let's begin with the orchestration of our machine learning pipelines. In [Chapter 2](#), we introduced DABs and examined the structure of the `databricks.yml` file. What we have not yet looked at are the resources that get deployed when the `databricks bundle deploy` command runs.

The `resources` folder is referenced in the bundle definition and includes the following files:

ml_pipeline.yml

This is a Lakeflow job that defines the whole ML pipeline.

ml_monitoring.yml

This is a Lakeflow job that updates the monitoring table.

alert.yml

This file contains alert-related resources.

In this chapter, we focus on the ML pipeline, and in [Chapter 6](#), we'll turn to monitoring and alerting resources.

Machine Learning Pipeline

Before we examine the structure of the `ml_pipeline.yml` file, let's clarify the objective. The file contains a Lakeflow job that brings together data preprocessing, model retraining and evaluation, and model deployment. [Figure 5-1](#) illustrates the pipeline's architecture, highlighting how it supports retraining and deploying models with built-in conditional logic.

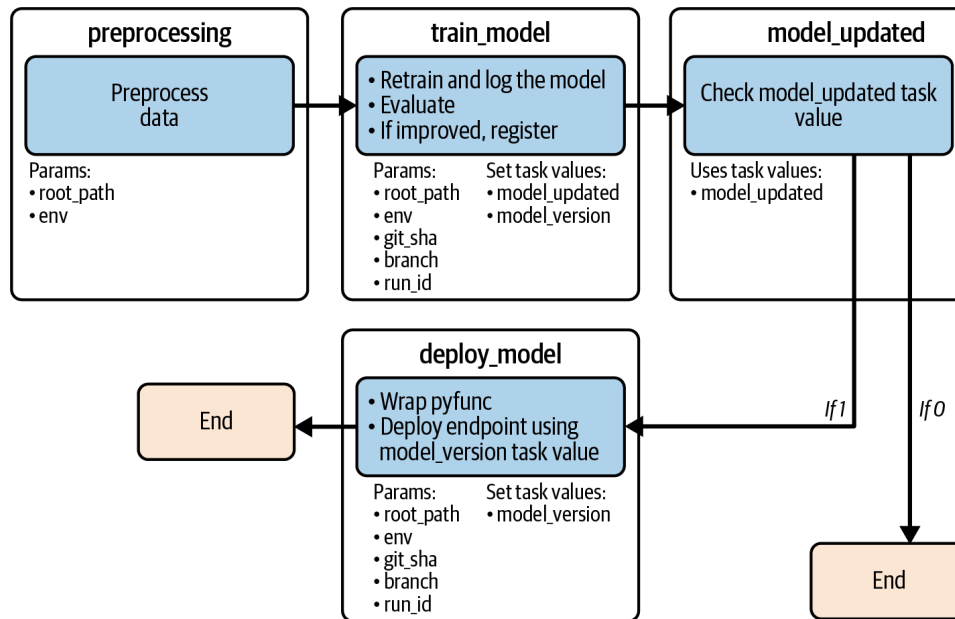


Figure 5-1. Lakeflow job structure

The workflow begins with a preprocessing step that prepares the data. Next, the `train_model` task retrains the model, evaluates its performance against the latest registered model on the new test set, and registers it if there is an improvement. It sets two task values: `model_updated` and `model_version`.

The `model_updated` task is an `IF` task in the Lakeflow job. It checks whether a new model was actually registered. If not (i.e., if the task returns `model_updated = 0`), the job ends early. If a new model was registered (i.e., if the task returns `model_updated = 1`), the workflow continues to the `deploy_model` task, which wraps the model as a PyFunc and deploys it to an endpoint using the latest `model_version`. This setup ensures that only evaluated and improved models are deployed to production.

Each task in the workflow (except the `IF` task) runs a Python script that requires specific parameters. Some parameters are set by the bundle:

`env`

This parameter matches the bundle target (e.g., `dev`, `prd`).

root_path

This is the root directory of the bundle.

git_sha and branch

These are variables that are defined in the bundle and overwritten by the CI/CD pipeline.

run_id, job_id, and task_name

These are set dynamically, using references like `{{job.run_id}}` and `{{task.name}}`, when the job starts.

We need these parameters for a few reasons:

- They allow the scripts to access the correct configuration files.
- We can use them to tag experiments, models, and deployments, so we can always trace a result back to the corresponding code and run.

To simplify parameter parsing, we'll define a `create_parser()` function in the `utils.common` module of our package.

First, we define the common arguments among all steps: `env` and `root_path`. We then create a `common` parser that accepts these two arguments, which is used for preprocessing and later for monitoring.

Next, the training and deployment steps require the same arguments, so we create a `train_deploy_model` parser for them:

```
def create_parser(args: Sequence[str] = None) -> argparse.Namespace:
    """Create and configure an argument parser for MLOps on Databricks.

    This function sets up a parser with subparsers for different MLOps operations

    :param args: Optional sequence of command-line argument strings
    :return: Parsed argument namespace
    """
    parser = argparse.ArgumentParser(description="Parser for MLOps on Databricks")
    subparsers = parser.add_subparsers(dest="command", required=True)

    # Common arguments
    common_args = argparse.ArgumentParser(add_help=False)
    common_args.add_argument(
        "--root_path", type=str, required=True,
        help="Path of root on DAB")
    common_args.add_argument(
        "--env", type=str, required=True,
        help="Path of env file on DAB")
```

```

# Data ingestion subparser
subparsers.add_parser("common", parents=[common_args], help="Common processor

# Model training & registering subparser
model_parser = subparsers.add_parser(
    "train_deploy_model",
    parents=[common_args],
    help="Model training and registering")
model_parser.add_argument(
    "--branch", type=str, required=True,
    help="branch of the project")
model_parser.add_argument(
    "--git_sha", type=str, required=True,
    help="git sha of the commit")
model_parser.add_argument(
    "--run_id", type=str, required=True,
    help="run id of the run of the Lakeflow job")
model_parser.add_argument(
    "--job_id", type=str, required=True,
    help="Lakeflow job id")
return parser.parse_args(args)

```

Using this approach means we don't have to define the argument parsing logic in each script, which simplifies the project code. Let's now take a deep dive into each of the steps.

Data Preprocessing

Let's take a look at the *ml_pipeline.yml* file. It has a resources section where one job, *ml-pipeline*, is defined. Once deployed, the job will be called *hotel-booking-ml-pipeline*. It runs based on a schedule (which we can set using [Quartz Cron syntax](#)): every Monday at 6:00 a.m. CET. We can set the pause status to *PAUSED* to nonproduction environments by using the *schedule_pause_status* bundle variable.

The schedule is illustrative: in practice, to choose the right cadence, you must think carefully about your specific use case. Retraining frequency depends on several factors. You should ask yourself how quickly your data distribution shifts. A recommendation system for a news platform might need daily updates as trending topics change rapidly, while an email classification model for internal processes might remain stable for months. Consider also the cost of retraining versus the cost of degraded performance: if a stale model means lost revenue or poor user experience, more frequent retraining pays for itself.

Triggering retraining only when monitoring detects performance degradation sounds appealing in theory. But in practice, this approach is often impractical. By the time your metrics have degraded enough to trigger retraining, you're already serving poor predictions to users. Moreover, the degradation itself may signal a deeper problem that retraining won't fix. If your input data quality has dropped, an upstream system changed its output format, or a key feature has stopped updating, throwing fresh training at the problem won't help. You'll just train a new model on the same broken data. But scheduled retraining with robust monitoring catches these issues earlier and keeps the system in a known, predictable state. We'll talk about monitoring in detail in [Chapter 6](#).

Here is how the schedule for the pipeline is defined in the corresponding *resources/ml_pipeline.yml* file:

```
resources:
  jobs:
    ml-pipeline:
      name: hotel-booking-ml-pipeline
      schedule:
        quartz_cron_expression: "0 0 6 ? * MON"
        timezone_id: "Europe/Amsterdam"
        pause_status: "${var.schedule_pause_status}"
```

We pass a usage policy ID (formerly known as a budget policy ID) for cost monitoring purposes:

```
budget_policy_id: "${var.usage_policy_id}"
```

We'll discuss cost allocation in [Chapter 9](#).

Here, we will use a serverless cluster to run the job. We'll define an environment called `default` and select environment version 4, which matches Python 3.12 (which we are using), as we discussed in [Chapter 2](#). Here, we'll also specify the project dependencies: all *.whl* files in the *dist* folder, where the project package built by the bundle is stored. This environment is then referenced in all the tasks:

```
environments:
  - environment_key: default
    spec:
      environment_version: "4"
```

```
dependencies:
  - ../dist/*.whl
```

The first task of the job is called `preprocessing`. It is a `spark_python` task that runs the `preprocess_data.py` Python script. Notice that the location of the script is relative to the location of the `ml_pipeline.yml` file. (It is located in the `resources` folder.):

```
tasks:
  - task_key: preprocessing
    environment_key: default
    spark_python_task:
      python_file: ../scripts/preprocess_data.py
      parameters:
        - "common"
        - "--root_path"
        - "${workspace.root_path}"
        - "--env"
        - "${bundle.target}"
```

You'll notice that the `preprocess_data.py` script takes two arguments: `env` and `root_path`. The first parameter, `common`, means that we will use the common parser to parse the arguments. The `databricks bundle deploy` command uploads all the files that belong to the bundle to a specific location (defined as the `root_path`). Our data preprocessing code depends on the `project_config.yml` file, which is stored in the `files` folder under the root path. Because the root path changes depending on the target, it is convenient to provide it as an argument. In the `project_config.yml` file, we define the environment-specific values for the catalog and schema. These values match the targets specified in the `databricks.yml` file, so we can pass the target as an argument.

Inside the `preprocess_data.py` script, we use the `create_parser` function we defined earlier to parse the arguments that were passed to the script. We need these arguments to identify the location of the configuration file and load it:

```
import pandas as pd
import yaml
from loguru import logger
from pyspark.sql import SparkSession

from hotel_booking.config import ProjectConfig
from hotel_booking.data.data_processor import DataProcessor
from hotel_booking.utils.common import create_parser
```

```

args = create_parser()

cfg = ProjectConfig.from_yaml(
    config_path=f"{args.root_path}/files/project_config.yml", env=args.env
)
logger.info("Configuration loaded:")
logger.info(yaml.dump(cfg, default_flow_style=False))

```

After that, the code gets really simple. We initialize the `DataProcessor` class we defined earlier by using the `Pandas DataFrame`, project configuration, and `SparkSession`. We preprocess the data, generate 1,000 synthetic observations, and store them in Unity Catalog under the catalog and schema defined in the `project_config.yml` file:

```

spark = SparkSession.builder.getOrCreate()

# Load the dataset
df = pd.read_csv(f"{args.root_path}/files/data/booking.csv")

# Initialize DataProcessor
data_processor = DataProcessor(df=df, config=cfg, spark=spark)
data_processor.preprocess()
if spark.catalog.tableExists(data_processor.target_table):
    data_processor.generate_synthetic_df(n=1000, max_date=None)

data_processor.save_to_catalog()

```

And with that, we're done with the data preprocessing step! Now, let's move to the next part: model training.

Training, Evaluating, and Registering a Model

In the `databricks.yml` file, we've defined the `branch` and `git_sha` variables, which are passed as parameters in the `train_model` task, along with other parameters. Here, we'll use the `train_deploy_model` parser we defined earlier:

```

- task_key: "train_model"
  environment_key: default
  depends_on:
    - task_key: "preprocessing"
  spark_python_task:
    python_file: "../scripts/train_register_model.py"
    parameters:
      - "train_deploy_model"

```

```

- "--root_path"
- "${workspace.root_path}"
- "--env"
- "${bundle.target}"
- "--branch"
- "${var.branch}"
- "--git_sha"
- "${var.git_sha}"
- "--run_id"
- "{{job.run_id}}"
- "--job_id"
- "{{job.id}}"

```

This task runs the `train_register_model.py` Python script. Here, we are mostly going through the same steps as in the “MLflow Evaluation” section of [Chapter 3](#). Just as in the data preprocessing step, we load the project configuration, initialize the `SparkSession`, and parse the arguments, and then, we use the `DataLoader` class to load and split the data into the train and the test set:

```

import mlflow
import yaml
from loguru import logger
from pyspark.sql import SparkSession

from hotel_booking.config import ProjectConfig, Tags
from hotel_booking.data.data_loader import DataLoader
from hotel_booking.models.lightgbm_model import LightGBMModel
from hotel_booking.utils.common import create_parser

args = create_parser()

cfg = ProjectConfig.from_yaml(
    config_path=f"{args.root_path}/files/project_config.yml", env=args.env)

logger.info("Configuration loaded:")
logger.info(yaml.dump(cfg, default_flow_style=False))

spark = SparkSession.builder.getOrCreate()

data_loader = DataLoader(spark=spark, config=cfg)
X_train, y_train, X_test, y_test = data_loader.split()

```

After we load the data, we train and log the model. The model metrics dictionary is stored as `metrics_new`. Notice that we pass actual values of the `run_id`, `git_sha`, and `branch` as tags:

```

model = LightGBMModel(config=cfg)
tags = Tags(**{"git_sha": args.git_sha, "branch": args.branch, "run_id": args.run_id})
model.train(X_train=X_train, y_train=y_train)
model_info = model.log_model(
    experiment_name="/Shared/hotel-booking-training",
    tags=tags,
    X_test=X_test,
    y_test=y_test,
    train_set_spark=data_loader.train_set_spark,
    train_query=data_loader.train_query,
    test_set_spark=data_loader.test_set_spark,
    test_query=data_loader.test_query,
)

metrics_new = model.metrics

```

Now, we'll evaluate the latest registered model (if it exists) on the new test set. If no older registered model with the `latest-model` alias exists, or if the newly trained model improves on the metric (here, we use root mean squared error as an example), we register it and set the `model_updated` and `model_version` task values, which we will use in the follow-up tasks:

```

sklearn_model_name = f"{cfg.catalog}.{cfg.schema}.hotel_booking_basic"
eval_data = X_test.copy()
eval_data[cfg.target] = y_test

should_register = True
try:
    mlflow.MlflowClient().get_model_version_by_alias(sklearn_model_name, "latest-model")
    model_exists = True
except mlflow.exceptions.RestException:
    model_exists = False
    logger.info(f"No {sklearn_model_name}@latest-model yet. Registering new model")

if model_exists:
    result = mlflow.models.evaluate(
        model=f"models://{sklearn_model_name}@latest-model",
        data=eval_data,
        targets=cfg.target,
        model_type="regressor",
        evaluators=["default"],
    )
    rmse_old = result.metrics["root_mean_squared_error"]
    rmse_new = metrics_new["root_mean_squared_error"]
    should_register = rmse_new < rmse_old
    logger.info(
        f"Current RMSE {rmse_old:.4f}, new RMSE {rmse_new:.4f} -> "
    )

```

```

        + ("registering new model" if should_register else "keeping current model"
    )

    if should_register:
        model_version = model.register_model(model_name=sklearn_model_name, tags=tags)
        dbutils.jobs.taskValues.set(key="model_version", value=model_version)
        dbutils.jobs.taskValues.set(key="model_updated", value=1)
    else:
        dbutils.jobs.taskValues.set(key="model_updated", value=0)

```

Deploying a Model Serving Endpoint

The `model_updated` task defines whether the pipeline will run further. It checks whether the `model_updated` task value is equal to 1: if the condition evaluates to true, the job proceeds to the final step, `deploy_model` task. It takes the same arguments as the `train_model` task:

```

- task_key: model_updated
  condition_task:
    op: "EQUAL_TO"
    left: "{{tasks.train_model.values.model_updated}}"
    right: "1"
  depends_on:
    - task_key: "train_model"

- task_key: "deploy_model"
  environment_key: default
  depends_on:
    - task_key: "model_updated"
      outcome: "true"
  spark_python_task:
    python_file: "../scripts/deploy_model.py"
    parameters:
      - "train_deploy_model"
      - "--root_path"
      - "${workspace.root_path}"
      - "--env"
      - "${bundle.target}"
      - "--branch"
      - "${var.branch}"
      - "--git_sha"
      - "${var.git_sha}"
      - "--run_id"
      - "{{job.run_id}}"
      - "--job_id"
      - "{{job.id}}"

```

The task runs the `deploy_model.py` script. After loading the project configuration and parsing the arguments, we retrieve the `model_version` task value from the `train_model` task. We then use it to look up the corresponding `ModelInfo` instance:

```
import mlflow
from loguru import logger

from hotel_booking.config import ProjectConfig, Tags
from hotel_booking.models.pyfunc_model_wrapper import HotelBookingModelWrapper
from hotel_booking.models.serving import serve_model
from hotel_booking.utils.common import create_parser

args = create_parser()

model_version = dbutils.jobs.taskValues.get(
    taskKey="train_model",
    key="model_version")
logger.info(f"Retrieved wrapped model version: {model_version}")

cfg = ProjectConfig.from_yaml(
    config_path=f"{args.root_path}/files/project_config.yml", env=args.env)

model_name = f"{cfg.catalog}.{cfg.schema}.hotel_booking_pyfunc"
wrapped_model_name = f"{cfg.catalog}.{cfg.schema}.hotel_booking_basic"

wrapped_model_info = mlflow.models.get_model_info(
    model_uri=f"models://{wrapped_model_name}/{model_version}"
)
```

We then use that to log and register the wrapper with its dependencies:

```
wrapper = HotelBookingModelWrapper()

with open(f"{args.root_path}/files/version.txt") as f:
    version = f.read().strip()
whl_name = f"hotel_booking-{version}-py3-none-any.whl"
code_paths = [f"{args.root_path}/artifacts/.internal/{whl_name}"]

tags = Tags(**{"git_sha": args.git_sha,
               "branch": args.branch,
               "run_id": args.run_id})

model_version = wrapper.log_register_model(
    wrapped_model_info=wrapped_model_info,
    pyfunc_model_name=model_name,
    experiment_name="/Shared/hotel-booking-pyfunc",
    code_paths=code_paths,
```

```
tags=tags,  
)
```

After we have registered the model, we can finally deploy it using the `serve_model` function that we defined earlier (and that you can find in the `models.serving` module) and finish our pipeline:

```
serve_model(  
    entity_name=model_name,  
    entity_version=model_version,  
    budget_policy_id=cfg.usage_policy_id,  
    endpoint_name="hotel-booking-pyfunc",  
    schema_name=cfg.schema,  
    catalog_name=cfg.catalog,  
    table_name_prefix="hotel_booking_monitoring",  
)
```

As mentioned earlier, we should view PyFunc more as a FastAPI model wrapper and should keep it separate from the model training logic. That way, we won't necessarily have to retrain it if the payload logic needs to change.

So far, we've been using *serverless compute* to run our jobs. Now, let's look at another Lakeflow job that explicitly defines a *classic cluster configuration* instead. In this example, we use *classic compute* purely for demonstration purposes, to show how cluster specifications work within a job definition. However, if you are working in the Databricks Free Edition, you should update the configuration to use serverless compute, as it is the only option available in that environment.

Serverless versus Classic Compute for Lakeflow Jobs

In the *resources* folder, you can find another *yml* file called *ml_monitoring.yml*, which defines the `hotel-booking-monitor-update` job.

This job is configured to run daily at 9:00 CET, and its schedule can be paused depending on the bundle deployment target (e.g., development, production). The job cluster, where we can set many different configurations, is defined, and we can use this cluster to run the `scripts/refresh_monitor.py` Python script:

```

resources:
  jobs:
    house-price-monitor-update:
      name: hotel-booking-monitor-update-workflow
      schedule:
        quartz_cron_expression: "0 0 9 * * ?"
        timezone_id: "Europe/Amsterdam"
        pause_status: "${var.schedule_pause_status}"
      tags:
        project_name: "hotel-booking"

    job_clusters:
      - job_cluster_key: "hotel-booking-cluster"
        new_cluster:
          spark_version: "17.3.x-scala2.13"
          data_security_mode: "SINGLE_USER"
          node_type_id: "r3.xlarge"
          driver_node_type_id: "r3.xlarge"
          num_workers: 1

    tasks:
      - task_key: "refresh_monitor_table"
        job_cluster_key: "hotel-booking-cluster"
        spark_python_task:
          python_file: "../scripts/refresh_monitor.py"
          parameters:
            - "common"
            - "--root_path"
            - "${workspace.root_path}"
            - "--env"
            - "${bundle.target}"
        libraries:
          - whl: ../dist/*.whl

```

Notice that we do not provide a usage policy ID in the job definition, as it can only be applied to serverless resources. Instead, we pass a tag for cost allocation.

We can also override the job's default cluster with an interactive cluster by specifying the `cluster_id` argument for a given target. For example, it's common practice to do this in the `dev` target to speed up iteration and debugging. In this setup, jobs run directly on an existing interactive cluster, rather than spinning up a new one. Additionally, we set the `dynamic_version` setting for `artifacts` to `true`, to ensure that the latest version of the package is always installed whenever the code changes, which is helpful during active development:

```
targets:
dev:
  default: true
  mode: development
  cluster_id: 0513-112814-kzixtb9s
  workspace:
    host: <your_host>
    root_path: "/Workspace/Users/${workspace.current_user.userName}/\
      .bundle/${bundle.target}/${bundle.name}"
  artifacts:
    default:
      type: whl
      build: uv build
      path: .
      dynamic_version: true
```

In [Chapter 6](#), we'll cover the monitoring job in detail, including its role in tracking model performance. For now, our focus is on how serverless jobs differ from jobs that utilize classic jobs compute, which can be useful if more customization is required.

So far, we've worked entirely in a development environment, without worrying about how code and models move across environments. But to address promotion strategies, we'll first need to understand the basics of Databricks platform design, including workspace structure, environment isolation, and compute options. That foundation is key for managing ML workflows in production.

Databricks Platform Design

We have already interacted with various components of the Databricks platform and experienced the Databricks workspace—a user interface that allows developers to create, access, and manage various Databricks assets, such as experiments, models, and jobs. It is common for organizations to have more than one workspace and use separate workspaces to isolate Databricks assets from other environments, teams, and even data and AI products.

We've also interacted with Unity Catalog: we used it to create Delta tables and register models. For a workspace to use Unity Catalog, it must be attached to a Unity Catalog metastore, which is the top-level container for metadata of all data and AI assets. You can have one metastore per cloud region, and you can only attach a workspace to one metastore in the

same cloud region. It is not uncommon for large organizations with international operations to have multiple Unity Catalog metastores.

In Unity Catalog, assets are organized using the following a three-tier hierarchy within the metastore:

1. Catalogs, which are the highest level of organization.
2. Schemas, which are contained within catalogs.
3. Assets, which include the tables, views, volumes, models, and functions within schemas. We refer to these assets with this three-part naming convention: *catalog.schema.asset*.

[Figure 5-2](#) depicts this hierarchy.

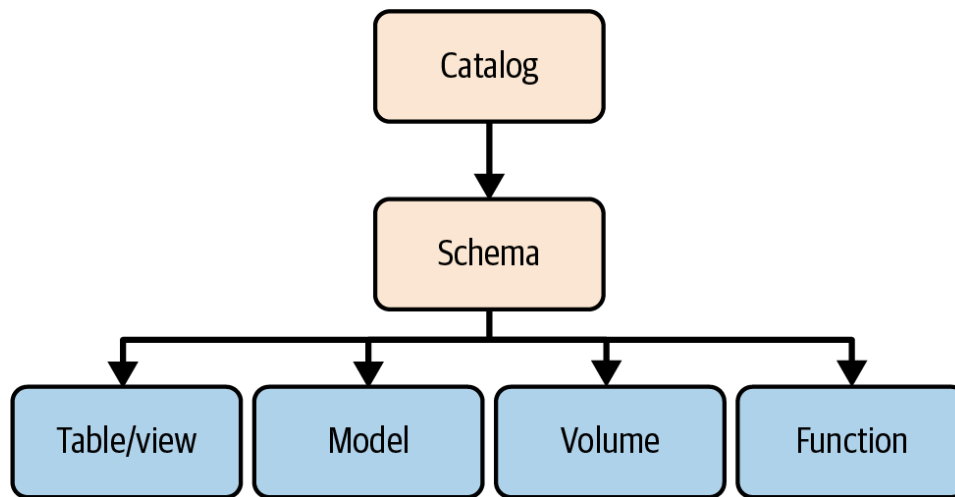


Figure 5-2. Unity Catalog metastore object hierarchy

All the workspaces belong to a Databricks account. Databricks account admin permissions are needed to create and manage workspaces and Unity Catalog metastores, and they're also needed to configure settings on the account level (such as by enabling certain features). [Figure 5-3](#) depicts the relationships among accounts, metastores and workspaces.

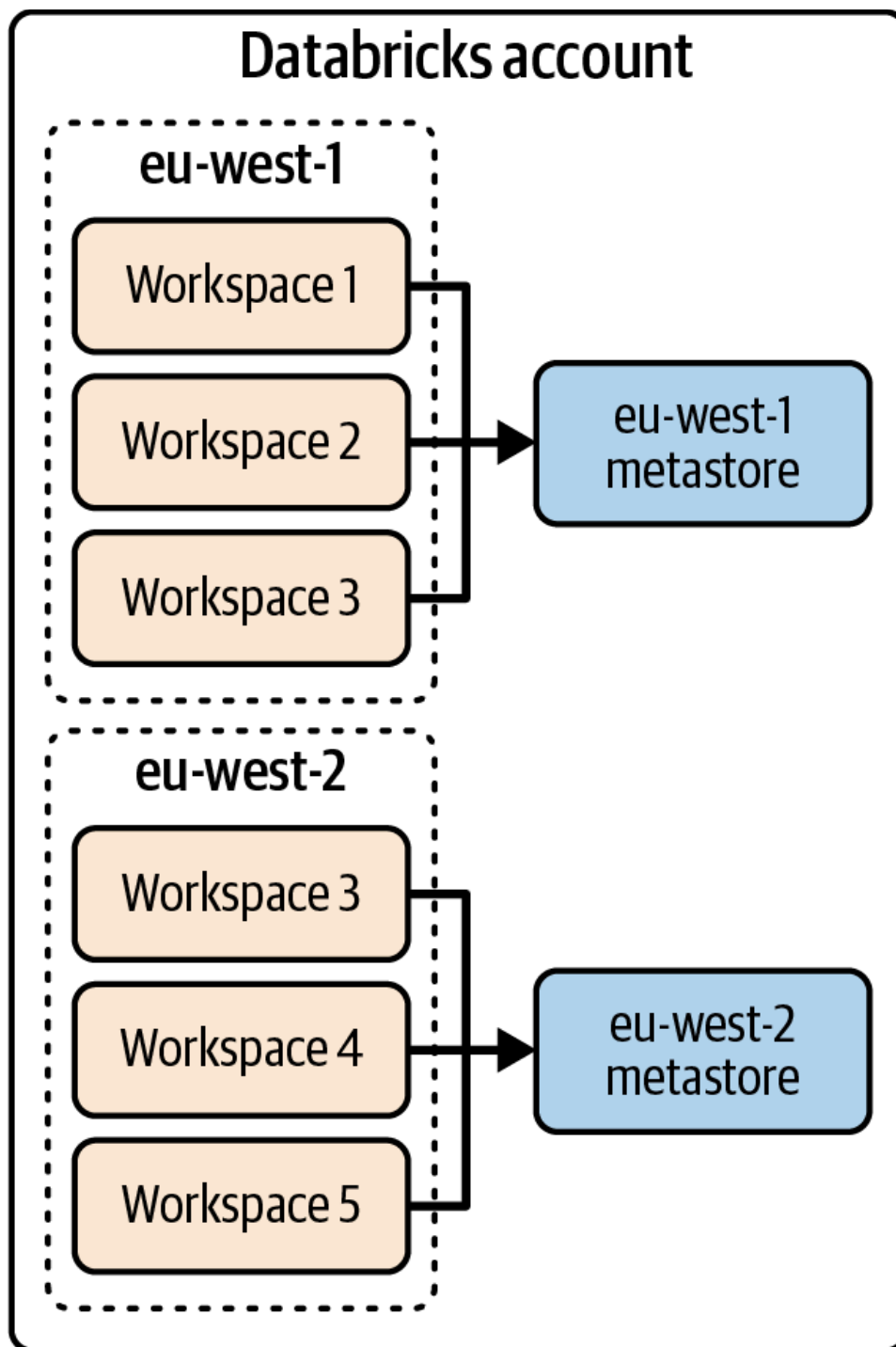


Figure 5-3. Relationships among accounts, workspaces and metastores

I've mentioned that you may want to have different Databricks workspaces for different environments, but how many environments do you actually need? Let's look at that next.

How Many Environments Do You Need?

The development, testing, acceptance, and production (DTAP) approach is probably the most common one in the industry.

One of the earliest formalizations of DTAP came from Cal Evans in 2009.¹ He noted that "The structure of development environments hasn't radically changed over the past 20 years." That means the DTAP pattern is

now over 35 years old, so it predates cloud computing, IaC, and the widespread adoption of version control.

Consider what the software development sequence looked like before DTAP emerged:

1. *Development* happened first, on individual laptops or desktops. Developers had access to a shared development server and database, which was often populated with a stale copy of production data. They'd manually copy files to the server via FTP or basic scripts. Version control existed via the Concurrent Versions System ([CVS](#)) or [Subversion](#), but it wasn't universal. Git was only invented in 2005.
2. *Testing* came next and was where code got integrated. On a predetermined schedule, the testing server would check out all the code, refresh the database, and execute testing scripts. Unit tests ran first, and integration and regression tests ran next to verify that all the pieces fit together and nothing that was previously working had gotten broken. (We'll talk about testing at the end of this chapter.) Quality assurance (QA) engineers owned this environment, but they had no access to production data.
3. *Acceptance* came after testing, and mirrored production in services and revision levels. This was where developers lost control: operations and release engineers took over. When code was ready for release, it was checked out onto the Acceptance server. Teams often re-ran integration and regression tests as a final sanity check. This was where the client actually saw and tested the code, using their own test plan.
4. *Production deployment* was the final phase. At the time, Cal optimistically wrote, "If all goes well, this should be a fully automated process—or, at least, a highly automated operation." In practice, it was anything but. Deployments happened in scheduled maintenance windows, often on weekends, with extensive sign-offs and manual steps.

The DTAP gates weren't just about catching bugs; they encoded organizational boundaries. Developers owned development and testing. Operations owned acceptance and production. Code was thrown "over the wall" between these silos.

This made sense in a world where the following things were true:

- Environments were expensive physical servers, which were impossible to reproduce reliably.
- Deployments were manual, risky, and infrequent.
- A bad deployment could mean hours or days of downtime, and roll-backs were slow.

- CI/CD tooling barely existed. ([Jenkins](#), then called Hudson, appeared in 2005 but wasn't mainstream.)

DTAP was a safety net for a world without better options. The organizational separation between dev and ops was a feature, not a bug: it forced deliberate handoffs and human verification at each stage.

But by the late 2000s, DevOps culture began challenging this model. If one team owned the entire cycle from development to production, did it really need four separate environments with formal handoffs among them?

Today, especially for ML/AI development, having three environments (development, acceptance, and production) is enough, for the following reasons:

- ML/AI development comes with certain requirements, such as access to production data in the development environment. This means we achieve a higher level of confidence that the system performs as expected at an earlier stage.
- IaC makes environments reproducible. We can deploy to development, acceptance, and production environments with identical configurations and security guidelines. This solves environment drift, which is one of DTAP's core problems.
- CI/CD is mature. Automated pipelines can build, test, and deploy code in minutes. The manual handoffs that DTAP formalized are now bottlenecks, not safeguards.
- Teams that follow DevOps principles and own code from development through production don't need organizational gates between environments.

However, you need to be aware that having four environments may still be necessary in regulated industries, such as critical infrastructure, finance, and health care, where multi-environment gates ensure safety for the users and serve as proof of compliance.

The Big Book of MLOps,² which was developed by Databricks, also mentions three environments: `dev`, `acc`, and `prd`. We use the three-environment setup in this book as well.

Platform Best Practices

Let's discuss an example of a Databricks platform setup. It's common for each individual product, a business unit, or a team to have a set of Databricks workspaces, one per environment— `dev`, `acc`, and `prd` —

and a schema or a set of schemas in environment-specific catalogs (`mlops_dev`, `mlops_acc`, and `mlops_prd`). [Figure 5-4](#) illustrates this structure.

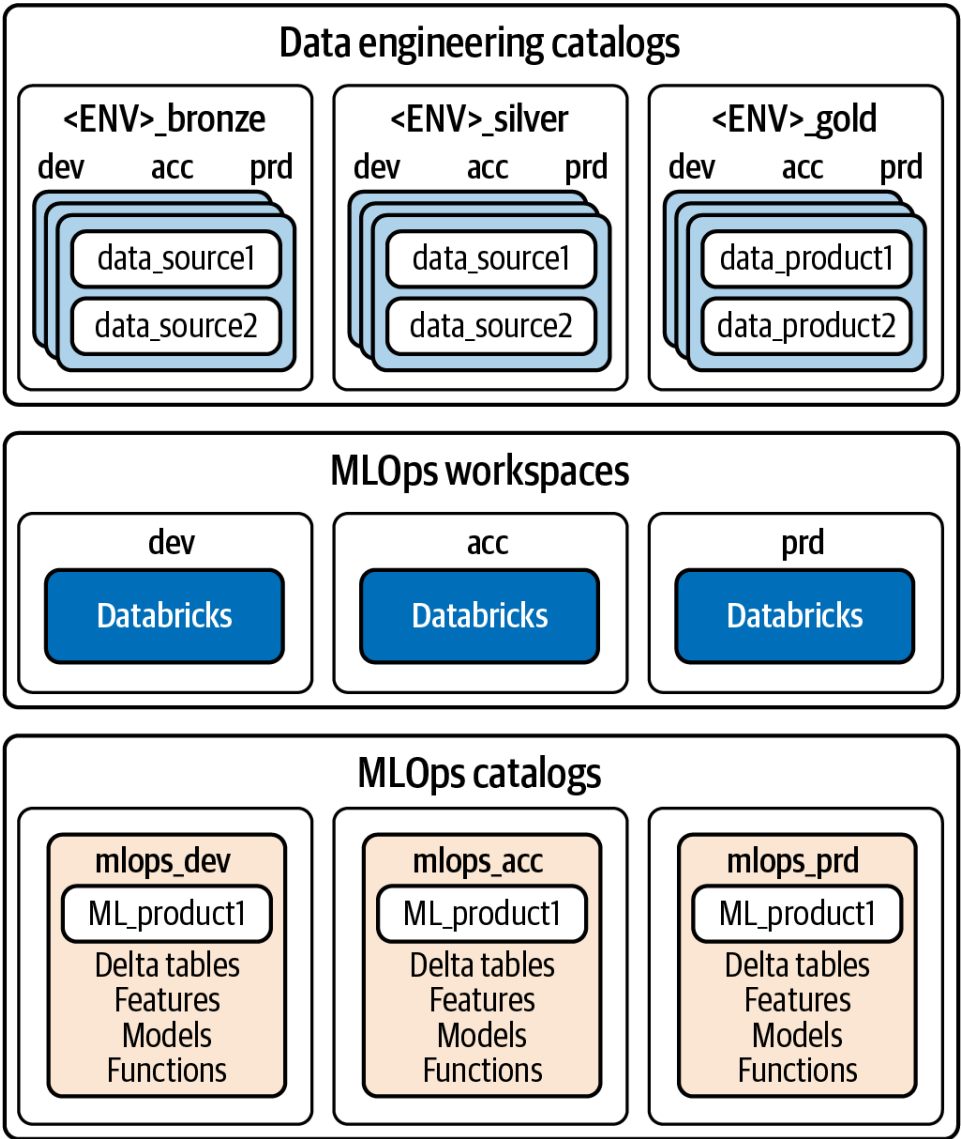


Figure 5-4. Example of Databricks platform setup for MLOps

ML teams act as both data consumers and data producers: they use curated datasets created by data engineering teams for training the models, and they generate outputs like predictions or embeddings that are consumed by downstream systems. The data that ML teams consume is stored in catalogs related to data products, while the data they produce is stored under ML product schemas in MLOps catalogs.

Data engineering teams commonly organize source data by using the medallion architecture, with separate catalogs for each environment and layer: `<env>_bronze`, `<env>_silver`, and `<env>_gold`:

Bronze

This layer holds raw, ingested data.

Silver

This layer includes cleaned and structured tables.

Gold

This layer contains business-ready datasets.

Each data source or product has a dedicated schema, which enables clear ownership and structured access control.

An important best practice is for the platform team to be able to read production data during development while strictly controlling where they can write data. In practice, ML teams should be able to read data product schemas from the production Gold catalog (`prd_gold`), even when working from the development workspace. This ensures that training and inference use the same data definitions and quality guarantees across all environments.

Lower-environment copies of data often differ in subtle but important ways (e.g., missing history, relaxed quality checks, schema drift). This can lead to issues that only appear in production, but using production Gold data for development significantly reduces these risks and helps minimize model drift.

At the same time, write access must remain environment specific. From each ML workspace (`dev` , `acc` , and `prd`), teams can write data only to their own schemas in an environment-specific ML catalog (`mlops_dev` , `mlops_acc` , or `mlops_prd`). This prevents accidental pollution of production assets, keeps environment promotion explicit, and maintains clear ownership boundaries.

For platform teams, this pattern balances consistency and safety. Data product teams retain ownership of Gold datasets in `prd_gold` , while ML teams consume them as read-only dependencies. All ML-generated artifacts are isolated in ML-owned catalogs. With Unity Catalog governance and read-only permissions, exposing production data for development is low risk and avoids the operational burden of maintaining multiple “production-like” data copies.

This structure enables modular, secure development across teams. Data engineers focus on building trusted data products, while ML teams build and deploy models independently on a consistent foundation of production-grade data.

I’ve just mentioned a number of things related to permissions, and that raises another important point: when you’re designing a Databricks platform, you need to have an access control strategy.

Access Control Strategy

In Databricks, access controls are applied at two distinct levels: the workspace level and the Unity Catalog level. Access is governed through securables—objects that can have permissions assigned to them. See [Figure 5-5](#) for a list of securables.

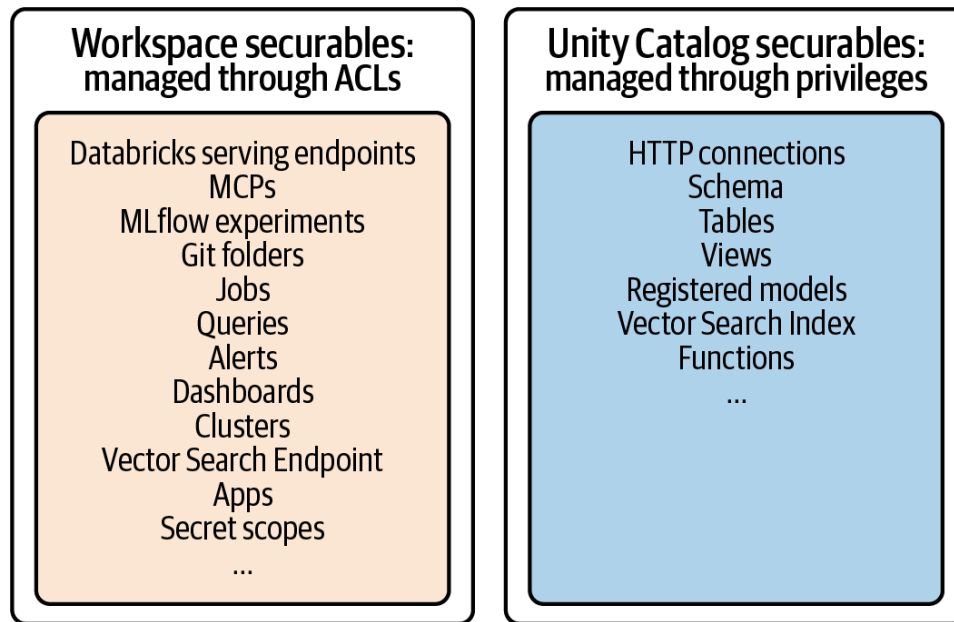


Figure 5-5. Workspace and Unity Catalog securables

Workspace securables include items like notebooks, dashboards, endpoints, clusters, and jobs, and access to them is managed within each individual workspace. *Unity Catalog securables* include catalogs, schemas, tables, and other data or AI assets, and access to them is defined at the metastore level.

Workspace securables are managed through access control lists (ACLs).³ Workspace admins have the `CAN_MANAGE` permission on all objects in their workspace (including clusters), while users have the `CAN_MANAGE` permission on objects they create.

Unity Catalog securables are managed through privileges.⁴ Metastore admins can create catalogs, manage permissions, and transfer ownership for any object, read and update the metadata of all objects in the metastore, and even delete a metastore. While metastore admins don’t have direct read or write access to data by default, they can grant themselves access by transferring ownership.

Consider a ML engineer working on two projects simultaneously: hotel booking pricing and demand forecasting. Each project has its own set of Databricks workspaces and Unity Catalogs. To prevent the engineer from accessing demand forecasting data while working in the hotel booking pricing workspace, you can use workspace binding.

Unity Catalog supports two access modes for securables: `OPEN` and `ISOLATED`. In `OPEN` mode, the securable can be accessed from any workspace. In `ISOLATED` mode, access is restricted to a specified list of workspaces.

Let's review how we can organize the permissions for a team working on the hotel booking pricing project. [Figure 5-6](#) outlines the permissions for the team and service principals on the workspace level.

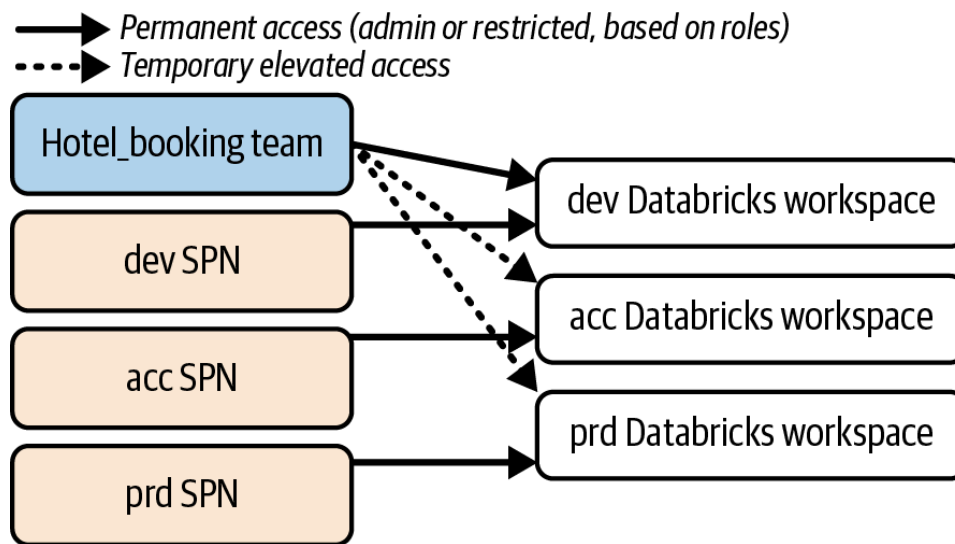


Figure 5-6. Project workspace permissions

The team should have permanent access to the development Databricks workspace, where most experimentation, model development, and collaboration happen. Workspace admin access should be limited to senior engineers or tech leads. Granting admin rights to all users is risky, as it enables unrestricted cluster creation and other unnecessary permissions. A better approach is to create a Databricks group and assign cluster policies that limit what kind of clusters that group can create. It is also important to keep in mind that this setup introduces collaboration challenges in development environments. The user who creates an object—such as a notebook, query, job, or Model Serving endpoint—must explicitly grant the appropriate permissions to the group so that other users can view, edit, or run shared assets.

Access to the acceptance (`acc`) and production (`prd`) workspaces should be restricted by default and only granted temporarily when needed, such

as for testing deployment pipelines or investigating production issues. This temporary elevated access minimizes risks while still supporting operational needs.

Additionally, service principals (SPNs) are used for automation and CI/CD pipelines. Each environment has a dedicated SPN with scoped permissions that are limited to its corresponding workspace. This separation ensures that automation processes only operate within their intended boundaries and follow the principle of least privilege.

[Figure 5-7](#) shows the permissions for the team and service principals on the Unity Catalog level.

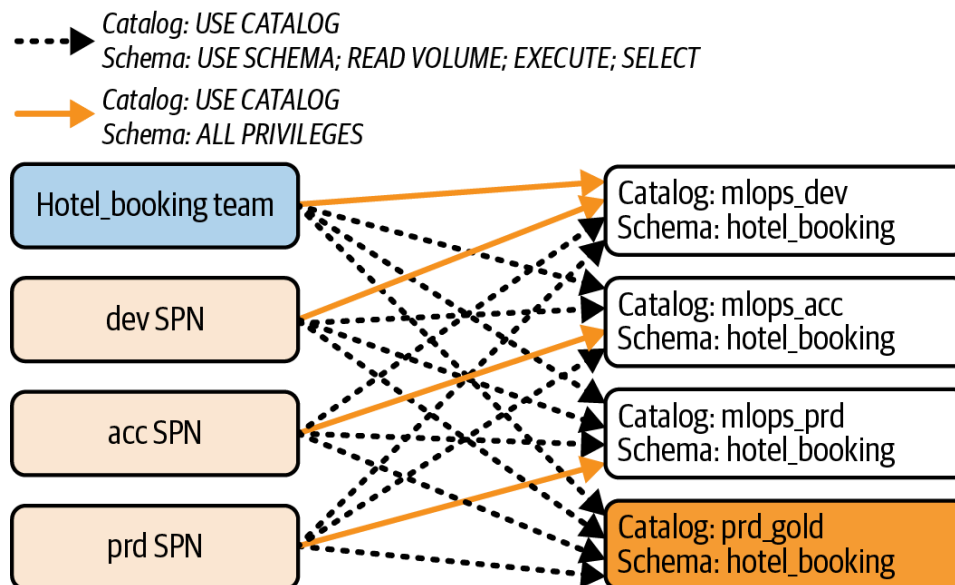


Figure 5-7. Project Unity Catalog privileges

The hotel booking team has `ALL_PRIVILEGES` permissions on the `mlops_dev.hotel_booking` schema. In other environments (`mlops_acc`, `mlops_prd`, and `prd_gold`), they have read-only access (`SELECT`, `READ_VOLUME`, and `EXECUTE`), which allows them safe use of production data for training and validation.

SPNs have `ALL_PRIVILEGES` access on schemas in the corresponding catalogs (`mlops_dev`, `mlops_acc`, and `mlops_prd`) and read-only access to `prd_gold.hotel_booking` to ensure consistent data consumption across environments, without risk of modifying production data. The privileges on the schema level are propagated to all underlying objects.

Promoting Code and Models to Higher

Environments

Now that we've covered how different environments (`dev` , `acc` , and `prd`) are structured and secured, we can look at how code and models move among them. First, let's talk about code promotion, which is tightly coupled with Git branching strategy. Let's explore how Git serves as the backbone of deployment in a multi-environment Databricks setup.

Git Branching Model

Anyone who uses version control has probably come across Gitflow, which is a branching model created by Vincent Driessen. After Driessen published the post titled “A Successful Git Branching Model” in 2010,^{[5](#)} Gitflow became the default branching strategy for many teams over the following years.

This model assumes two default branches, development and main, along with multiple long-lived feature and release branches. This structure often leads to unnecessary complexity and a lot of branches to manage (see [Figure 5-8](#)).

1. A feature branch `feature/foo` is created from the `main` branch. After a number of commits, the branch is ready.
2. A developer creates a pull request (PR).
3. At PR to the `main` branch, a CI pipeline run gets triggered.
4. It runs pre-commit checks and unit tests. An approval is required to merge the pull request.
5. When the feature branch is merged to `main`, a CD pipeline run gets triggered and targets two environments: acceptance and production.
6. The CD pipeline in the acceptance environment can run directly, but deployment to the production environment should be approved by a reviewer first. Usually, some manual checks on the acceptance environment are required to make sure everything works as expected.
7. [Figure 5-9](#) provides a simplified illustration of this GitHub flow approach.

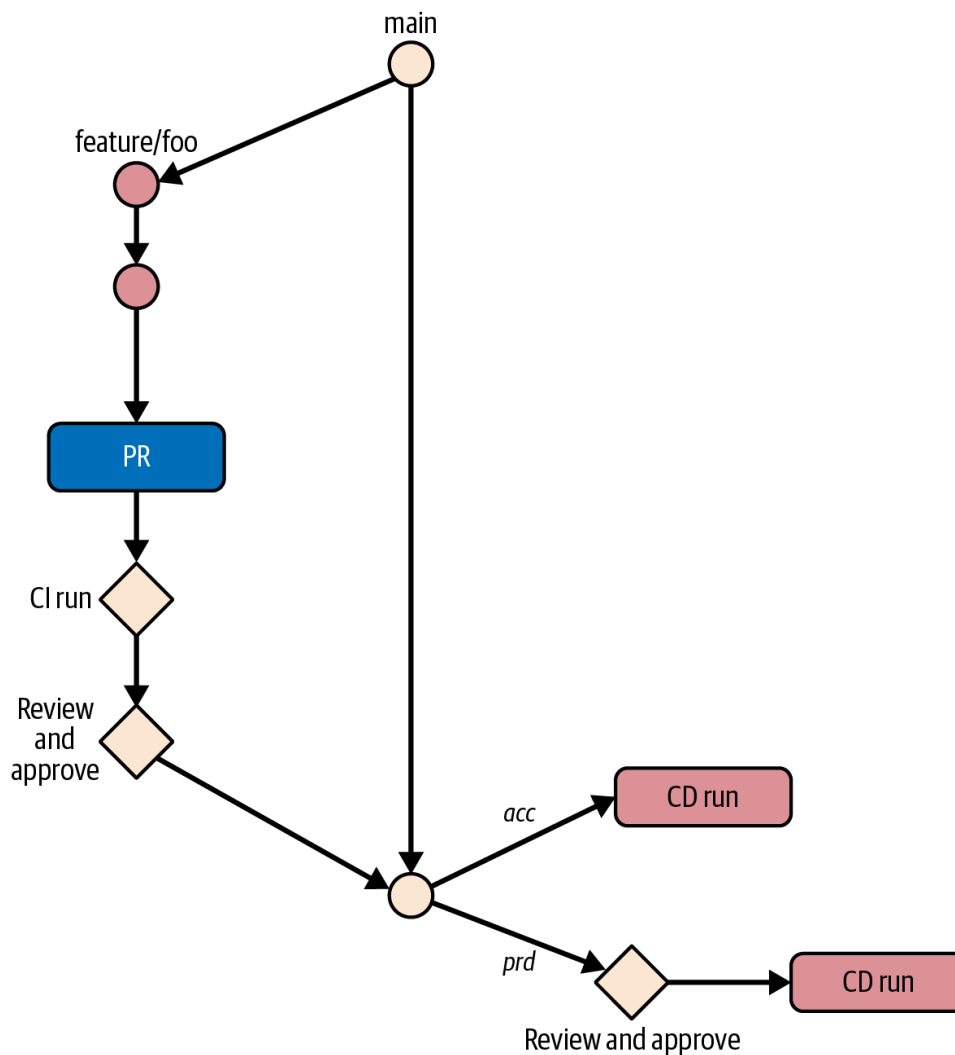


Figure 5-9. Simplified GitHub flow

Let's now take a close look at the CI and CD pipelines, which I mentioned previously when explaining the Git branching strategy.

Continuous Integration Pipeline

Let's take a look at the continuous integration (CI) pipeline for the MLOps part of the book (from the `ci_mlops.yml` file in the `.github/workflows` folder), which is triggered to run on each PR to the main branch if files in the `mlops` folder have changed:

```
on:
  pull_request:
    branches:
      - 'main'
    paths:
      - 'mlops/**'
```

The pipeline has one job. The first step in this job is to check out the repository under GitHub workspace, so the workflow can access the code. Notice that we fetch the history of all branches and tags by setting `fetch_depth` to `0` (the default is `1`), to make sure that the next step can run correctly:

```
jobs:
  pytest_and_checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@de0fac2e4500dabe0009e67214ff5f5447ce83dd #v6.0.2
        with:
          # Fetch all history for all branches and tags
          fetch-depth: 0
          ref: ${ github.head_ref }
```

The following steps install `uv` and all the dependencies, and they run pre-commit checks and PyTest:

```
- name: Install uv
  uses: astral-sh/setup-uv@0cec208311dfd045dd5311c1add060b2062131d57 #v8.0.0

- name: Install the dependencies
  run: |
    uv sync --extra ci

- name: Run pre-commit checks
  run: |
    uv run pre-commit run --all-files

- name: run pytest
```

```
run: |  
    uv run pytest
```

It's also important to note that we do not use tags but instead commit hash for public GitHub Actions. This is a good security practice that minimizes the risk of malicious attacks, which can happen if the tag gets overwritten by a malicious actor.⁷

Continuous Deployment Pipeline

Let's take a look at the continuous deployment (CD) pipeline, which is triggered to run on push to the main branch if files in the `mlops` folder have changed:

```
on:  
  push:  
    branches:  
      - 'main'  
  paths:  
    - 'mlops/**'
```

This pipeline defines a single deployment job with a matrix strategy, which allows the same job definition to run across multiple environments using variable combinations. In this case, the job runs twice (once each in the `acc` and `prd` environments), based on the values defined in the matrix.

GitHub environments are used to manage environment-specific variables, secrets, and federated credentials under a consistent name. By referencing `${{ matrix.environment }}`, the job dynamically applies the appropriate configuration for each run.

The job requires specific permissions:

id-token

This is write permission to fetch an OpenID Connect (OIDC) token for authentication.

contents

This is write permission to push a Git tag after a successful production deployment.

The `DATABRICKS_HOST` environment variable is set using the GitHub environment-specific variable with the same name:

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        environment: [acc, prd]
    environment: ${ matrix.environment }
    permissions:
      id-token: write # to fetch openID token
      contents: write # to push tag
    env:
      DATABRICKS_HOST: ${ vars.DATABRICKS_HOST }
```

The first step checks out the repository under GitHub workspace:

```
steps:
  - name: Checkout Source Code
    uses: actions/checkout@de0fac2e4500dabe0009e67214ff5f5447ce83dd #v6.0.2
```

In this example, we use Amazon Web Services (AWS), so we configure AWS authentication and retrieve Databricks credentials from AWS Secrets Manager. We could have used a similar process for Azure or Google Cloud Platform (GCP). This setup enables secure access to Databricks during deployment. Note that in such a case, our cluster specification would need modification as well.

The second step uses the `configure-aws-credentials` GitHub action to assume an identity and access management (IAM) role defined in the `ARN_ROLE` variable and then set the AWS region. This grants the pipeline temporary credentials to access AWS resources.

The third step fetches secrets from AWS Secrets Manager by using those credentials. The secret contains two key-value pairs:

`DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET`. These values are exported as environment variables with the same names, which are then used to authenticate with Databricks when running `databricks bundle deploy`. Note that the Databricks CLI requires three environment variables for authentication with the service principal: `DATABRICKS_CLIENT_ID`, `DATABRICKS_CLIENT_SECRET`, and `DATABRICKS_HOST`:

```

- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials
    @ec61189d14ec14c8efccab744f656cffd0e33f37 #v6.1.0
  with:
    role-to-assume: ${ vars.ARN_ROLE }
    aws-region: eu-west-1

- name: Fetch secrets
  uses: aws-actions/aws-secretsmanager-get-secrets
    @2cb1a461cbd4865ac4299648312e4704c646cd53 #v3.0.1
  with:
    secret-ids: |
      , ${ vars.SECRET_ID }
    parse-json-secrets: true

```

The final steps in the pipeline prepare the deployment environment and handle promotion logic for production. First, the Databricks CLI is installed using the official setup action. This is required to run the `databricks bundle deploy` command, which publishes the bundle definition to the target environment.

Next, the `uv` package manager is installed. It's used to build any Python packages defined in the bundle configuration. Finally, the deployment step runs `databricks bundle deploy`, passing the current `git sha` as a variable. Note that deploying a bundle does not trigger any jobs. Instead, jobs will run based on defined triggers within the Databricks workflow:

```

- name: Install Databricks CLI
  uses: databricks/setup-cli@2260866f83a41a2df55e1cfe7ffe038b78325bf6 #v0.295.0
  with:
    version: 0.295.0

- name: Install uv
  uses: astral-sh/setup-uv@0cec208311dfd045dd5311cladd060b2062131d57 #v8.0.0

- name: Deploy to Databricks
  env:
    DATABRICKS_BUNDLE_ENV: ${ matrix.environment }
  run: databricks bundle deploy --var="git_sha=${ github.sha }"

```

In our CD pipeline example, we have authenticated with AWS using federated credentials that are set up for each GitHub environment. However, before we can do that in the real world, we need to create a set of service

principals on Databricks and a set of IAM roles, policies, and secrets on AWS.

OpenID Connect

[Figure 5-10](#) shows an example of the setup. It assumes that we have three different Databricks workspaces (`dev` , `acc` , and `prd`). Each of these workspaces contains a service principal, and the client secret and client id of these service principals are stored in the respective secrets in AWS Secret Manager. We also need to create custom IAM policies that allow us to read these secrets from the secret manager. These policies are attached to the respective IAM roles, which are assumed when CI/CD pipelines get executed under their respective environments.

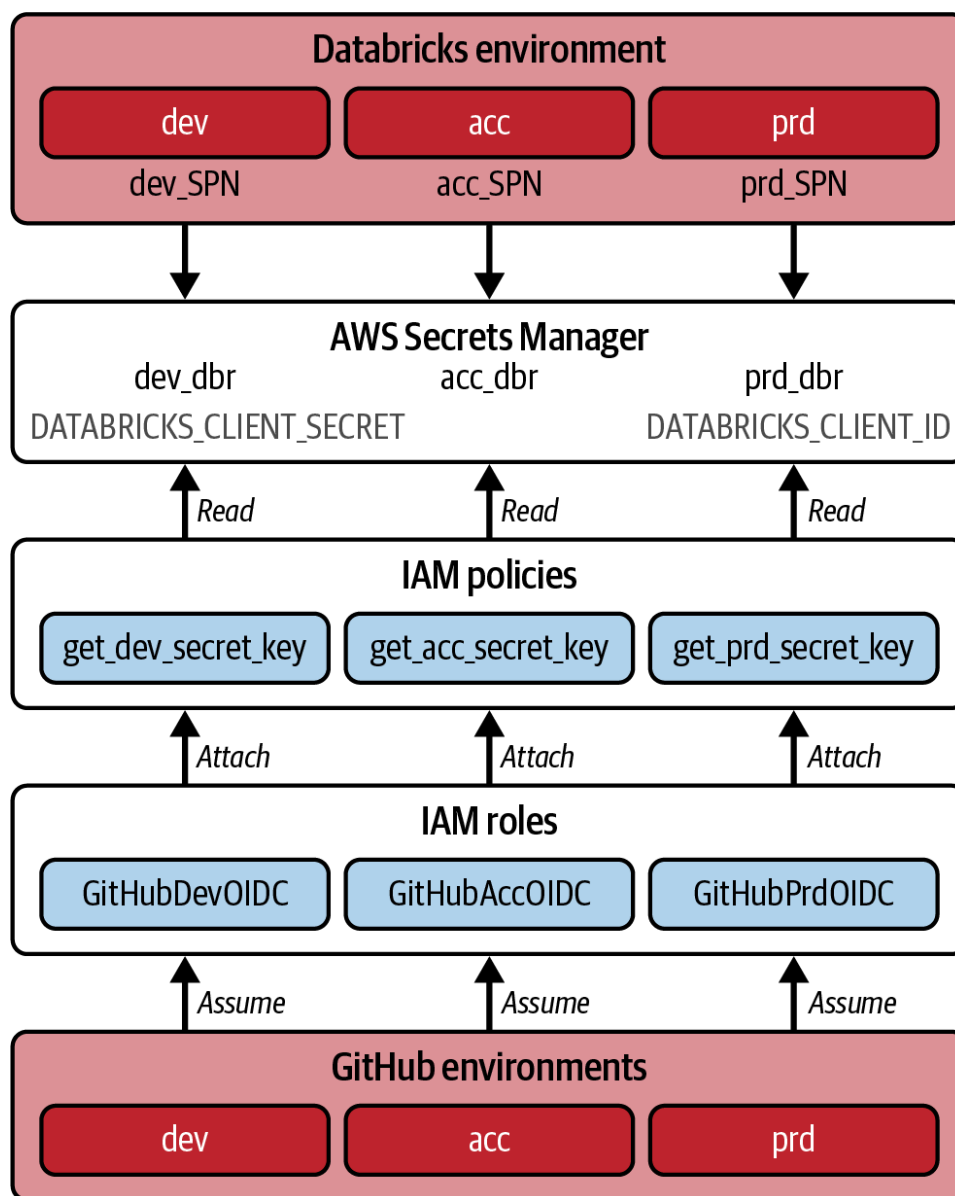


Figure 5-10. IAM roles and policies

You can find the trust policy definition for the IAM role. Please note that you must create an OpenID Connect identity provider by setting

provider to `token.actions.githubusercontent.com` and
audience to `sts.amazonaws.com`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated":
"arn:aws:iam::ACCOUNT_ID:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        },
        "StringLike": {
          "token.actions.githubusercontent.com:sub":
            "repo:ORG_NAME/REPO_NAME:environment:ENVIRONMENT_NAME"
        }
      }
    }
  ]
}
```

In our setup, we do not use GitHub secrets. Instead, we use the OIDC setup, which is generally considered to be a more secure solution. However, we use environment variables that contain different values in each environment. [Figure 5-11](#) shows the view of configured environments in GitHub.

Environments New environment

You can configure environments with protection rules, variables, and secrets. [Learn more about configuring environments.](#)

prd	3 variables	
acc	3 variables	
dev	3 variables	

Figure 5-11. Environments in GitHub repository

Here are example values for the variables in the development environment:

- `ARN_ROLE` : `arn:aws:iam::123456789:role/GitHubDevOIDC`
- `DATABRICKS_HOST` : `https://dbc-c2e8445d-159d.cloud.databricks.com`
- `SECRET_ID` : `dev_dbr`

Using CI/CD pipelines alone is not sufficient to ensure secure deployments. When I described the GitHub flow, I mentioned the approval step multiple times. We can configure this step by using the branch and deployment protection rules to enforce a manual review before code is merged or deployed. Let's dive into it.

Branch and deployment protection rules

Implementing branch protection rules is essential for preventing developers from committing directly to important branches, like main. GitHub offers rich functionality for these rules, but some of them are more critical than others. While features like requiring status checks to pass are nice to have, I believe the absolute minimum for any project is to enable the “Require a pull request before merging” rule, along with the sub-options shown in [Figure 5-12](#).

Protect matching branches

☒ **Require a pull request before merging**
 When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.

☒ **Require approvals**
 When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.

Required number of approvals before merging: 1 ▼

☒ **Dismiss stale pull request approvals when new commits are pushed**
 New reviewable commits pushed to a matching branch will dismiss pull request review approvals.

☒ **Require review from Code Owners**
 Require an approved review in pull requests including files with a designated code owner.

☐ **Restrict who can dismiss pull request reviews**
 Specify people, teams, or apps allowed to dismiss pull request reviews.

☐ **Allow specified actors to bypass required pull requests**
 Specify people, teams, or apps who are allowed to bypass required pull requests.

☒ **Require approval of the most recent reviewable push**
 Whether the most recent reviewable push must be approved by someone other than the person who pushed it.

Figure 5-12. Recommended minimal branch protection rules

These include the following:

- Requiring at least one (ideally, two) approvals
- Requiring a review if a new commit is pushed after the pull request is approved
- Using the CODEOWNERS file: automatically assigning reviewers based on the code being changed

- Enforcing approval: requiring the most recent push to be approved by someone other than the person who pushed it

I also recommend enabling the “Do not bypass the above settings” option to ensure these rules cannot be circumvented.

I mentioned earlier that deployment to production should be approved by a reviewer. We can enable this by using the deployment protection rules that are selected in [Figure 5-13](#), which we can set for each environment and repository. These rules provide necessary guardrails without making the CD pipelines more complicated than they need to be.

Environments / Configure prd

Deployment protection rules

Configure reviewers, timers, and custom rules that must pass before deployments to this environment can proceed.

The screenshot shows the 'Deployment protection rules' configuration page for the 'prd' environment. It features several sections: 'Required reviewers' (checked), 'Add up to 6 more reviewers' (with a search input), 'Prevent self-review' (checked), 'Wait timer' (unchecked), and 'Enable custom rules with GitHub Apps' (with a 'Preview' badge and links to learn more or create rules).

☒ **Required reviewers**
Specify people or teams that may approve workflow runs when they access this environment.

Add up to 6 more reviewers

Search for people or teams...

☒ **Prevent self-review**
Require a different approver than the user who triggered the workflow run.

☐ **Wait timer**
Set an amount of time to wait before allowing deployments to proceed.

Enable custom rules with GitHub Apps Preview
[Learn about existing apps](#) or [create your own protection rules](#) so you can deploy with confidence.

Figure 5-13. Deployment protection rules

So far, we’ve explored key considerations for promoting code across environments to ensure secure and reliable deployment. We assumed two things: models are retrained independently in each environment, and re-training is stateless, starting from scratch every time. But this isn’t always realistic.

For deep learning models, which can be large and expensive to train, it’s often preferable to resume training from previously saved weights. In these cases, you typically want to train the model once, often in a development environment, and promote it without retraining. However, copying models between Unity Catalogs (e.g., from `dev` to `acc` to `prd`) is not the best approach. Instead, it is more efficient to use a shared Unity Catalog, where the model is stored once and promotion is handled through aliases and approval workflows, rather than physical duplication. Let’s discuss how you can accomplish this.

Promoting Models to Higher Environments

Let's say the ML team is working on updating the weights for the hotel booking embedding model in the Databricks development workspace. The resulting model is registered in the central Unity Catalog, and it should eventually be deployed behind a Model Serving endpoint in the Databricks production workspace. [Figure 5-14](#) demonstrates how to do this in a secure way.

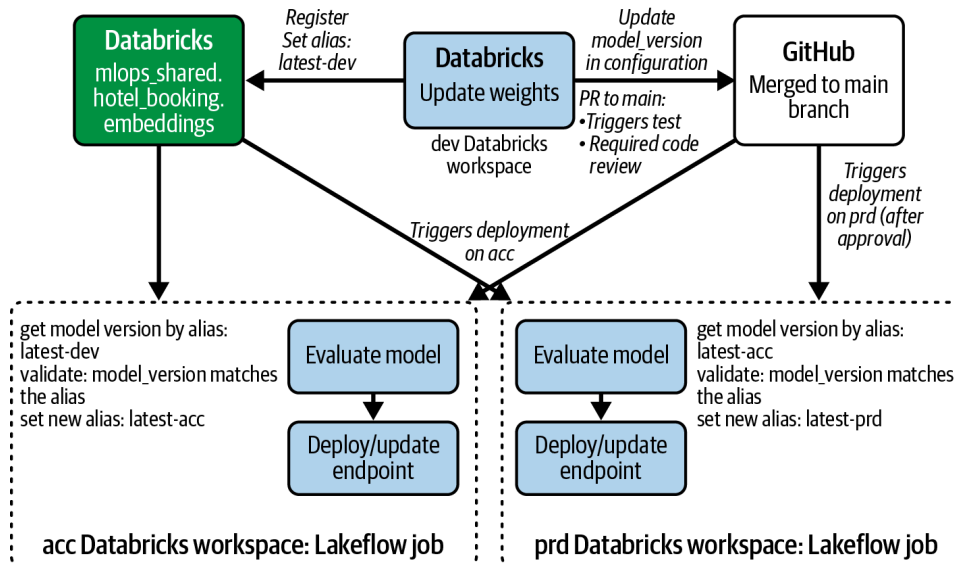


Figure 5-14. Promoting a model to higher environments

In this workflow, the embedding model is managed centrally in the `mlops_shared.hotel_booking.embeddings` catalog and promoted across environments using MLflow aliases. The model is large, so it cannot be trained locally. After updating weights in the development workspace, the model is registered and assigned the `latest-dev` alias.

In the simplest scenario, the model update did not require the code change, but we still need to store references to the updated model version in the configuration file (for example, `project_config.yml`). A PR to the main branch gets created, and it triggers tests and requires code review. After the code is merged to the main branch, the CD pipeline gets triggered, and deployment to acceptance and production gets executed there. Deployment to production is protected and requires approval. Earlier in the chapter, we looked at the CD pipeline, which only updates the Lakeflow job definition, but it can also trigger the execution of the Lakeflow job.

In the acceptance environment, the Lakeflow job gets the model version by `latest-dev` alias, verifies that the model version matches the one defined in the configuration file, and evaluates the model. For embeddings, programmatic evaluation typically involves computing similarity scores or ranking quality using predefined benchmarks or labeled pairs. If the evaluation confirms that the model performs as expected, the alias is updated to `latest-acc` and the model is deployed to the acceptance endpoint. A similar process follows in the production workspace, where we get the model version by `latest-acc` alias, assuming that the pipeline ran in the acceptance environment first.

In the approach we've discussed, the model stays in a shared catalog, and promotion across environments is handled through aliases and code promotion. This is gated by a manual approval step, rather than by duplicating the model. This setup enables efficient, auditable, and environment-aware model lifecycle management.

Earlier, when we built the CI pipeline, testing showed up as a single step. It is easy to treat this as a checkbox and move on, but it deserves a closer look. What actually belongs in that step, and why ML systems ask more of it than ordinary software does, is the subject of the next section.

Testing

Many ML practitioners have converged on the same observation: production incidents frequently originate from data quality issues, schema changes, feature pipeline failures, training-serving skew, deployment mistakes, and monitoring gaps, rather than flaws in the underlying model architecture.

Jeremy Jordan, a core maintainer of PyTorch Lightning, explains what goes into effective testing in his blog.⁸ Developing ML models relies on a large amount of traditional software development: processing data, building features, orchestrating training, and exposing interfaces to external systems. Effective testing of ML systems therefore requires both a traditional software testing suite for the surrounding infrastructure and a model testing suite for the trained artifacts.

In this section of the book, we limit ourselves to practical considerations, since testing could be a whole book on its own. You can find test examples in the repository that accompanies this book.

ML Model Testing

Model tests are divided into pre-train tests and post-train tests:

Pre-train tests

These run before training and short-circuit a wasted job: the output shape matches the labels, class probabilities sum to one, a single gradient step decreases the loss, and there is no label leakage between the training and validation sets.

Post-train tests

These interrogate the behavior the model actually learned. They include the following kinds of tests:

Invariance tests

These are tests for perturbations that must not change the prediction.

Directional expectation tests

These are tests for perturbations that must change the prediction in a known direction (e.g., more bathrooms should not lower a predicted house price).

Minimum functionality tests

These are tests on critical data slices.

It is also important not to confuse model evaluation with model testing:

Model evaluation

This summarizes performance and tells you whether the model is good on average. We performed model evaluation earlier in this book, before registering a model for deployment.

Model testing

This consists of explicit checks for specific behaviors the model must exhibit.

The distinction between these two matters, because an evaluation can improve overall while introducing a regression or bias on an important slice of data.

There are also tests for data specifically: assertions about datasets, schemas, null and range constraints, and distribution expectations. On Databricks, [DQX](#) provides declarative checks of this kind, and the same idea extends to monitoring the input tables that feed your processing pipelines.

We won't go deep into any of these here. They sit on top of a more fundamental layer. The rest of this section focuses on software engineering tests and their Databricks-specific considerations, because that foundation is where ML teams most often underinvest. For concrete model testing examples, read Eugene Yan's blog.⁹

Software Testing

For decades, software engineering has organized tests around the *test pyramid*: there are many fast unit tests at the base, fewer integration tests in the middle, and a small number of end-to-end tests at the top. [Figure 5-15](#) shows the test pyramid on Databricks that follows the same idea. We'll explain the details later.

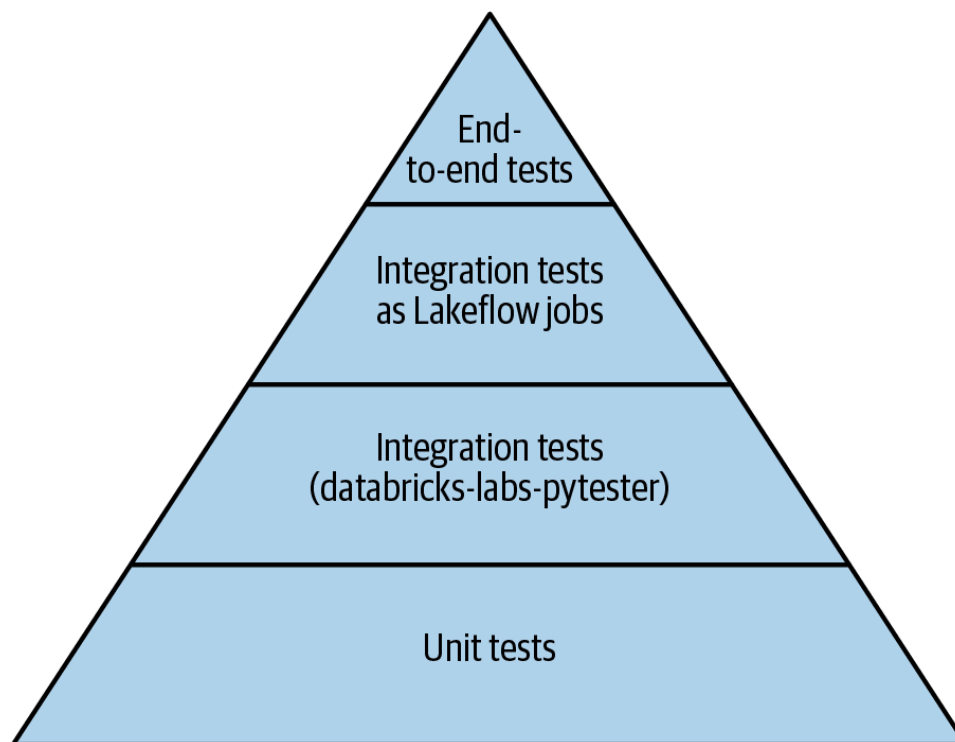


Figure 5-15. Test pyramid on Databricks

The trade-off is simple: as tests become more realistic, they also become slower, more expensive, and harder to debug. The goal is to catch everything you can at the cheapest layer and reserve higher-level tests for what only they can verify. The same model applies to ML systems on Databricks.

There is a lot of great literature on software testing. These are the ones we particularly recommend:

- [*Effective Testing: From Slop to Software*](#) by Matt Harrison (self-pub.)
- [*Python Testing with pytest*](#) by Brian Okken (O'Reilly)

Let's go through the layers one by one, but not in too much detail, as whole books are devoted to this topic.

Unit testing

A *unit test* isolates a small piece of logic and runs entirely locally: with no Databricks workspace, no cluster startup, no Unity Catalog access, and no network calls. These are the cheapest tests you have: they run in milliseconds, give deterministic feedback, and execute on every commit. They catch obvious mistakes early, and with AI-assisted development tools, writing them is practically free. There is no longer any excuse to skip them.

Your code does use actual SDK clients, and since we do not want to use them, we need to mock. Python's `unittest.mock` and `pytest-mock` make this straightforward.

You should mock only at genuine system boundaries such as REST APIs, SDK clients, and storage systems, and you should use real objects for code within your application. Excessive mocking couples tests with implementation details and produces suites that stay green while the system is broken.

Two rules prevent most mistakes:

- Patch where the name is looked up, not where it is defined. Patch the import used by the module under test, not the module that defines the object.
- Use `autospec=True` whenever possible. A mock with `autospec` derives its interface from the real object, so if a method is renamed, removed, or given different parameters, the test fails immediately. Without it, mocks accept attributes and signatures that no longer exist, so they produce green tests over broken code.

Databricks considerations

For Databricks projects, we reserve unit tests for logic that's expressible independently of the platform: data transformations on plain Python values, configuration parsing, prompt construction, request validation, and

business rules. If a test needs a running cluster, a workspace, or a catalog, it belongs in the integration layer.

Spark transformations are where unit testing on Databricks becomes complicated. Ideally, we would test them against a local Spark session that's running entirely in memory. In a standard PySpark project, this is straightforward: we install `pyspark`, create a local `SparkSession`, and run transformations against small test DataFrames.

The complication is that `databricks-connect` and `pyspark` are mutually exclusive. Once `databricks-connect` is installed, local Spark sessions become unavailable. Also, `databricks-connect` is pulled in by the version of `databricks-agents` we use in the book.¹⁰

One way out is to keep these dependencies out of the project's core library and install them only in the job environment that executes the workload. We can then test the transformation code locally with `pyspark`. This works, but it requires discipline and makes the dependency setup more error-prone. We take the simpler route: we accept that all Spark execution goes through `databricks-connect`, which brings us to integration testing.

Integration testing

There are two levels of integration testing on Databricks:

- The first level *runs Spark logic on Databricks compute*. Once Spark execution goes through `databricks-connect`, tests that exercise Spark logic run remotely rather than locally. The `spark` fixture from `databricks-labs-pytester` is exactly this: a Databricks Connect-backed session, rather than a local one. It still isolates a single transformation, uses small datasets, and gives fast feedback, but its execution is remote. These tests run against remote Spark on serverless compute, execute quickly, and can be launched directly from an IDE. They occupy the role that local Spark unit tests play in a standard PySpark project.
- The second level *validates an entire pipeline end to end*: workflow orchestration, task dependencies, parameter flow, permissions, and persistence to Unity Catalog. A typical pattern is to create temporary schemas, trigger the deployed job, assert on the resulting tables, and clean up afterward.
- At this level, the job definition itself is part of the system under test. Many failures originate not in the code but in the configuration: there may be a missing wheel, an incorrect `environment_key`, a broken

task dependency, or a parameter wired to the wrong task. When pipelines are defined through DABs, these failures are easy to test. The `databricks bundle validate` test catches definition errors before execution starts, and deployment to an isolated target is itself a test. A failed deployment is a failed test.

Taras Chaikovsk describes integration testing patterns on Databricks in detail in the Databricks community technical blog.^{[11](#)}

Practical considerations: Integration tests in CI

The orchestrated ML pipeline can run as part of CI, on a small data sample, to confirm that the whole thing is wired together. A run that samples data, assembles features, and then trains, evaluates, registers, and deploys a model exercises every wiring decision in a way no isolated test can.

The obstacle is duration. Even on sampled data, a run takes long enough that you do not want a GitHub runner to wait until the job finishes. The pattern that avoids this is asynchronous (see [Figure 5-16](#)).

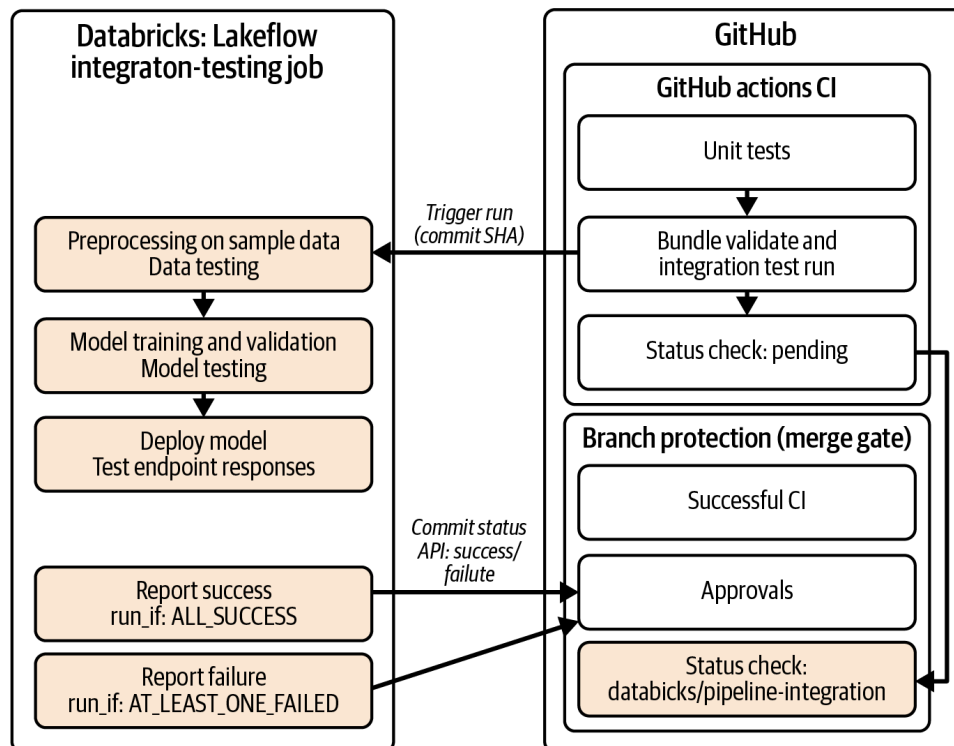


Figure 5-16. Testing pattern with GitHub Actions and Lakeflow jobs

The CI workflow validates the bundle, deploys it to the test target, triggers the job run, sets the commit status to pending through the GitHub [Commit Status API](#), and exits green. The commit SHA is passed in as a job parameter so the run can report back against the right commit.

A commit status can be `error`, `failure`, `pending`, or `success`, and it will surface on any PR involving the commit. A commit status can include `target_url` and `description`: the URL points to the Databricks run output, the description summarizes what happened, and both make the status useful in the GitHub UI. The `context` field names the service that is reporting the status (for example `databricks/pipeline-integration`), so this check stays distinct from other checks on the same commit. The `repo:status` OAuth scope grants access to statuses without granting access to repository code.

The run reports its own results through two tasks at the end of the job. One runs with `run_if: ALL_SUCCESS` and posts `success`. The other runs with `run_if: AT_LEAST_ONE_FAILED` and posts `failure`. Splitting it in two is what makes failure reportable: a single final task runs only on full success, so a failure anywhere would leave the commit stuck on `pending`. Each task knows its outcome from the condition that triggered it, so neither has to inspect the run. Both reporters depend on every pipeline task, not just the last one, so any task that fails is a direct dependency failure the condition can see. The full lifecycle is three writes: `pending` at trigger time and then exactly one of `success` or `failure`.

Branch protection turns that status into a merge gate. You should mark the status context as a required status check. The pull request stays `pending` until a reporter fires, and merging is blocked until the check is green.

Notice that [Figure 5-16](#) also lists data tests, model tests, and endpoint payload tests that confirm the output is what we expect. These can run as part of the Lakeflow job. The first level of integration testing, which consists of the Spark tests on Databricks compute, runs in the CI pipeline alongside the unit tests, since CI already authenticates to Databricks to trigger the job.

End-to-end testing

Integration tests prove that the pipeline works. End-to-end tests prove that the system works, including the consumers downstream.

The model is served behind an endpoint, and a frontend team, a mobile app, or another service consumes it. The contract between the model and its consumer is a real interface that consists of the request format, response schema, latency expectations, error handling, and fallback behav-

ior. It breaks in production like any other interface. End-to-end tests validate it from the consumer's perspective.

The common failure mode looks like this:

1. The ML team tests the pipeline, evaluates the model, and tests the endpoint in isolation.
2. An incident still happens because the frontend expected one field and the endpoint returned another, or latency doubled after a model update, or the consumer cannot handle a class label the retrained model now outputs.

None of this is specific to ML. It is the contract testing that any service architecture needs. However, ML teams skip it disproportionately because the boundary between the model team and the product team is broader than usual, and “It’s done” gets defined as “The model is served.”

These tests run in an acceptance environment where a real model is served and either a real consumer or a realistic test harness for it calls the endpoint. These tests are slow and require coordination across teams, so they sit at the top of the pyramid: you need a few of them, but you cannot skip them.

End-to-end tests and production monitoring are two points on a continuum, rather than separate boxes. An end-to-end test validates the consumer contract at a point in time in a controlled environment. Monitoring validates the same contract continuously against real traffic. For ML systems, with their delayed labels and gradually shifting prediction distributions, some of the most important validation can only happen in production. A/B testing, which we briefly covered earlier, is one form of validation that runs against live traffic. Monitoring is the subject of the next chapter.

Conclusion

This chapter moved from model development to a structured deployment strategy built on CI/CD and DABs. The Databricks platform design provides the foundation for reliable and secure ML operations across environments, from development to acceptance to production.

We defined end-to-end ML pipelines declaratively in *databricks.yml*. Configuring jobs as code and passing parameters to Python tasks gives us

traceability, portability, and automation, which are the key enablers of MLOps. The same bundle definition runs in every environment.

We also covered testing for ML systems, from unit tests on individual transformations, through integration tests that prove the pipeline runs, to the end-to-end tests that validate the consumer contract. The pyramid tells you how to expend effort, conducting many fast tests at the bottom and a few slow ones at the top, but skipping none of them.

In the next chapter, we'll shift our focus to monitoring, which is the final piece that closes the loop by ensuring models stay reliable once deployed.

- ¹ Cal Evans, “Professional Programming: DTAP—Part 1: What Is DTAP?,” *php[architect]*, July 27, 2009, <https://oreil.ly/Jg9yM>
- ² Databricks, *The Big Book of MLOps*, 2nd ed. (Databricks, 2023), <https://oreil.ly/r5fYY>.
- ³ Consult [the Databricks documentation](#) for more details on ACLs.
- ⁴ Consult [the Databricks documentation](#) for more information on privileges.
- ⁵ Vincent Driessen, “A Successful Git Branching Model,” January 5, 2010, <https://oreil.ly/HzKrc>.
- ⁶ The note of reflection is published in the same blog where the original idea was described.
- ⁷ For more details, see [an article about the GitHub Actions Worm written by Palo Alto Networks](#).
- ⁸ Jeremy Jordan, “Effective Testing for Machine Learning Systems,” August 19, 2020, <https://oreil.ly/4m1E>.
- ⁹ Check out [his blog post on testing machine learning code and systems](#).
- ¹⁰ `databricks-agents` removed `databricks-connect` from its requirements in version 1.9.1. If you pin 1.9.1 or later, the conflict does not arise and you can skip this workaround.
- ¹¹ Taras Chaikovsk, “Integration Testing for Lakeflow Jobs with Pytest and Databricks Connect,” *Databricks Community Technical Blog*, December 2025, <https://oreil.ly/hSI23>.

Previous chapter

< [4. Model Serving: Architectures and Implementation](#)

Next chapter

[6. Monitoring ML Applications](#) >